

Natural Language Processing — Revision Handbook

TG CPGET 2026 Exam Preparation

This handbook covers all 8 topics from your syllabus: Language Processing & Python, Text Corpora & Lexical Resources, Processing Raw Text, Categorizing & Tagging Words, Learning to Classify Text, Deep Learning for NLP, Extracting Information from Text, and Analysing Sentence Structure.

Each section has: **Concept (simple language) → Key Definitions/Formulas → Worked Examples → Practice Questions.**

TABLE OF CONTENTS

1. Language Processing and Python
 2. Accessing Text Corpora and Lexical Resources
 3. Processing Raw Text
 4. Categorizing and Tagging Words
 5. Learning to Classify Text
 6. Deep Learning for NLP
 7. Extracting Information from Text
 8. Analysing Sentence Structure
 9. Quick Revision Formula Sheet
 10. Mixed Mock Test (10 Questions)
-

1. Language Processing and Python

1.1 Concept in Simple Language

NLP is the branch of AI concerned with making computers understand, interpret, and generate human language. Python is the preferred language for NLP because of libraries like **NLTK** (Natural Language Toolkit), which gives ready-made tools for tokenizing, tagging, parsing, and analysing text.

Think of text as raw, messy material (like cotton) and NLP as the spinning mill that turns it into usable thread (structured data) — tokens, tags, trees, meanings.

Core built-in Python text structures for NLP:

- **String** - sequence of characters (`("Hyderabad")`)
- **List** - ordered, mutable collection (`(['NLP', 'is', 'fun'])`)
- **Tuple** - ordered, immutable collection

- **Dictionary** – key-value pairs, used for frequency counts, word-tag mapping

NLTK basic pipeline: Raw Text → Tokenization → Tagging → Parsing → Semantic Analysis

1.2 Key Definitions

Term	Definition
Corpus	A large, structured collection of texts used for linguistic analysis
Token	A single unit of text — usually a word or punctuation mark
Tokenization	Splitting text into tokens
Type	A distinct/unique word form in a text (vocabulary item)
Lexical Diversity	$\frac{\text{Number of Types}}{\text{Number of Tokens}}$ — measures vocabulary richness
Frequency Distribution	A mapping of each word to the number of times it occurs
Concordance	Shows every occurrence of a word with surrounding context
Collocation	A sequence of words that occur together more often than by chance (e.g., "strong tea")

Key Formula:

$$\text{Lexical Diversity} = \frac{\text{Total Number of Distinct Words (Types)}}{\text{Total Number of Words (Tokens)}}$$

(Value close to 1 → highly diverse vocabulary; close to 0 → repetitive text)

1.3 Worked Examples

Example 1: Tokenize the sentence "NLP is powerful, and Python makes it easy." → Tokens:

`['NLP', 'is', 'powerful', ',', 'and', 'Python', 'makes', 'it', 'easy', '.']` (10 tokens, punctuation counted separately)

Example 2: Compute lexical diversity for text: "the cat sat on the mat the cat ran"

- Tokens = 9 (the, cat, sat, on, the, mat, the, cat, ran)
- Types = {the, cat, sat, on, mat, ran} = 6
- Lexical Diversity = $6/9 = 0.67$

Example 3: Using `FreqDist` on words `['dog', 'cat', 'dog', 'bird', 'cat', 'dog']` → `FreqDist = {dog:3, cat:2, bird:1}`; most common word = "dog"

Example 4: `nltk.Text.concordance('love')` on a novel returns every line containing "love" with a few words of context on each side — used to study word usage/sense in context.

Example 5: Bigrams of `['I', 'love', 'NLP']` → `[('I', 'love'), ('love', 'NLP')]` (used to detect collocations like "machine learning").

1.4 Practice Questions

1. Lexical diversity of a text with 500 tokens and 125 types is: (a) 0.25 (b) 4.0 (c) 125 (d) 500 — (Ans: a)
 2. Which NLTK class is used to obtain word frequency counts? (a) Text (b) FreqDist (c) ConcordanceIndex (d) Tokenizer
 3. Which of these is NOT a token in the sentence "Mr. Rao went to Hyderabad."? (a) Mr. (b) Rao (c) went (d) Hyderabad, Mr — choose the odd one out and justify.
 4. A bigram is a sequence of how many consecutive words?
 5. Differentiate between "type" and "token" with one example.
 6. Which Python data structure is best suited to store word→frequency mappings, and why?
 7. In NLTK, `text.collocations()` is used to find: (a) rare words (b) frequently co-occurring word pairs (c) POS tags (d) parse trees
 8. Compute lexical diversity: tokens = 200, types = 50.
 9. What is the primary purpose of a concordance view in text analysis?
 10. State two advantages of using Python (over Java/C++) for NLP tasks.
-

2. Accessing Text Corpora and Lexical Resources

2.1 Concept in Simple Language

A **corpus** is a body of text (e.g., Gutenberg texts, Brown Corpus, Reuters). NLTK provides ready-made corpora and tools to access them by category, file, or sentence. **Lexical resources** are structured collections of word-related information — dictionaries, thesauri, and especially **WordNet**, which organizes words into sets of synonyms (**synsets**) linked by semantic relations.

Imagine a corpus as a library of books, and a lexical resource as a dictionary/thesaurus sitting on the reference desk that tells you the meaning and relationships of words used in those books.

2.2 Key Definitions

Term	Definition
Corpus (pl. Corpora)	Collection of text/speech samples used for linguistic research

Term	Definition
Categorized Corpus	Corpus organized into categories, e.g., Brown Corpus (news, fiction, humor)
Stopwords	Common function words (the, is, at) usually removed before analysis
WordNet	A lexical database of English organizing words into synsets with semantic links
Synset	A set of synonymous words representing one concept
Hypernym / Hyponym	Hypernym = more general term ("animal" for "dog"); Hyponym = more specific term ("dog" for "animal")
Meronym / Holonym	Meronym = part of ("wheel" of "car"); Holonym = whole containing part ("car" for "wheel")
Lemma	The canonical/dictionary form of a word within a synset
Conditional Frequency Distribution (CFD)	Frequency distribution computed separately for each condition/category

2.3 Worked Examples

Example 1: `nltk.corpus.gutenberg.fileids()` lists files like `'austen-emma.txt'` — used to access raw text of Jane Austen's *Emma*.

Example 2: Using WordNet: `wn.synsets('car')` returns synsets like `car.n.01` (automobile), showing "car" can mean automobile, railway carriage, gondola, etc. (polysemy).

Example 3: Hypernym path for "dog" → `dog → canine → carnivore → mammal → animal → organism`.

Example 4: Stopword removal: Original: "This is the best book I have read"; After removing stopwords (this, is, the, i, have): `['best', 'book', 'read']`.

Example 5: Conditional Frequency Distribution on Brown Corpus categories (news vs. romance) shows the word "could" occurs more in fiction/romance than in news — demonstrating genre-based word usage.

2.4 Practice Questions

- WordNet primarily organizes words based on: (a) alphabetical order (b) frequency (c) semantic relations (synsets) (d) length
- The relation between "leaf" and "tree" is best described as: (a) hypernym (b) hyponym (c) meronym (d) synonym

3. Which corpus is organized into categories like news, editorial, and fiction? (a) Gutenberg (b) Brown Corpus (c) Reuters (d) WordNet
 4. Define "synset" with one example.
 5. What does `wn.synsets('bank')` illustrate about the word "bank"?
 6. A word that is more general in meaning compared to another is called its _____.
 7. Why are stopwords typically removed before text analysis? Give two reasons.
 8. What is the purpose of a Conditional Frequency Distribution?
 9. Differentiate between a "lemma" and a "synset."
 10. Name any two lexical resources available in NLTK besides WordNet.
-

3. Processing Raw Text

3.1 Concept in Simple Language

Raw text (from files, web pages, or PDFs) is unstructured and messy — it contains HTML tags, punctuation, inconsistent case, and encoding issues. **Processing raw text** means cleaning and converting it into a usable form: tokenizing, normalizing case, removing noise, stemming/lemmatizing, and segmenting into sentences.

Think of raw text like unwashed vegetables — before cooking (analysis), you must wash, peel, and chop them (clean, tokenize, normalize).

3.2 Key Definitions

Term	Definition
Encoding	The scheme (e.g., UTF-8, ASCII) used to represent characters as bytes
Regular Expression (Regex)	A pattern used to search/match specific character sequences in text
Tokenization	Splitting text into words/sentences
Stemming	Crude chopping of word endings to get a root form (may not be a real word) — e.g., Porter Stemmer: "studies" → "studi"
Lemmatization	Reducing a word to its dictionary/base form using vocabulary and grammar rules — e.g., "studies" → "study"
Normalization	Converting text to a consistent form (lowercase, removing punctuation)
Segmentation	Dividing text into sentences (sentence segmentation) or words

Key distinction:

Stemming: fast, rule-based, may produce non-words (e.g., "caring" → "car")
Lemmatization: slower, dictionary-based, always produces valid words (e.g., "caring" → "care")

3.3 Worked Examples

Example 1: Porter Stemmer on "running", "runs", "ran" → "run", "run", "ran" (note: "ran" is irregular and may not stem correctly — a known stemmer limitation).

Example 2: Lemmatization: "better" (adjective) with WordNet Lemmatizer using POS='a' → "good"; without POS info → "better" (unchanged) — shows importance of POS tagging in lemmatization.

Example 3: Regex `(\w+)` applied to "NLP-2026 is here!" → matches

`['NLP', '2026', 'is', 'here']` (splits on non-word characters).

Example 4: Sentence segmentation: "Dr. Rao works at IIT. He teaches NLP." → Naive splitting on "." incorrectly breaks after "Dr." — this is why NLTK's `sent_tokenize` uses trained models (Punkt) rather than simple rules.

Example 5: Case normalization: "The Quick Brown Fox" → "the quick brown fox" (helps in avoiding duplicate counting of "The" and "the" as different words).

3.4 Practice Questions

1. Stemming of the word "studies" using Porter Stemmer typically gives: (a) study (b) studi (c) studie (d) studies
2. Which technique guarantees a valid dictionary word as output? (a) Stemming (b) Lemmatization (c) Tokenization (d) Segmentation
3. Regex `(\d+)` matches: (a) words (b) whitespace (c) one or more digits (d) punctuation
4. What problem can naive sentence segmentation face with abbreviations like "Dr." or "Mr."?
5. Differentiate stemming and lemmatization with an example each.
6. Which NLTK sentence tokenizer uses a pre-trained unsupervised model?
7. Why is case normalization important before frequency analysis?
8. What is the output of applying `(\w+)` regex on "co-operate_2026"?
9. Explain why encoding issues (like UTF-8 vs ASCII) matter in text processing.
10. State two real-world applications where raw text cleaning is a critical first step.

4. Categorizing and Tagging Words

4.1 Concept in Simple Language

Part-of-Speech (POS) tagging assigns a grammatical category (Noun, Verb, Adjective, etc.)

to each word in a sentence based on its definition and context. This is essential because the same word can act differently depending on context (e.g., "book" can be a noun or verb).

4.2 Key Definitions

Term	Definition
POS Tagging	Assigning a grammatical tag to each word (NN=Noun, VB=Verb, JJ=Adjective, RB=Adverb)
Tagset	The standardized set of tags used (e.g., Penn Treebank tagset)
Rule-Based Tagger	Uses hand-written rules based on word/context patterns
Stochastic Tagger	Uses probability (e.g., HMM) based on training data
N-gram Tagger	Assigns tag based on the previous N-1 tags/words (Unigram, Bigram, Trigram Tagger)
Backoff Tagger	Combines multiple taggers — if one fails, falls back to a simpler one
Default Tagger	Assigns the same (most frequent) tag to every word — baseline tagger

Common Penn Treebank Tags:

NN - Noun, singular	NNS - Noun, plural
VB - Verb, base form	VBD - Verb, past tense
JJ - Adjective	RB - Adverb
PRP - Personal pronoun	IN - Preposition
DT - Determiner	CC - Coordinating conjunction

Tagging Accuracy Formula:

$$\text{Tagging Accuracy} = (\text{Number of Correctly Tagged Words} / \text{Total Number of Words}) \times 100$$

4.3 Worked Examples

Example 1: "The dog barks loudly." → Tags: The/DT dog/NN barks/VBZ loudly/RB

Example 2: Ambiguity: "I will book a ticket" (book = VB) vs. "I read a book" (book = NN) — same word, different POS depending on context.

Example 3: Unigram tagger trained on Brown Corpus tags each word with its most frequent tag seen during training, regardless of context — fast but context-blind.

Example 4: Backoff chaining: `Trigram → Bigram → Unigram → Default('NN')` — if the trigram tagger cannot tag a word (unseen sequence), it "backs off" to bigram, then unigram, then defaults to NN.

Example 5: Default tagger assigns 'NN' to every word in "Run fast now" → Run/NN fast/NN now/NN — clearly inaccurate but useful as a fallback/baseline.

4.4 Practice Questions

1. In Penn Treebank tagset, "VBD" denotes: (a) Verb, base form (b) Verb, past tense (c) Verb, gerund (d) Verb, 3rd person singular
 2. A tagger that assigns the most frequent tag to every word regardless of context is called: (a) Bigram Tagger (b) Default Tagger (c) HMM Tagger (d) Rule-based Tagger
 3. What is a "backoff tagger" and why is it used?
 4. Give an example of a word that can have two different POS tags depending on context.
 5. Which tagset is most commonly used in NLTK for English POS tagging?
 6. A Bigram Tagger uses information from how many preceding word(s)/tag(s)?
 7. Tagging accuracy = 180 correctly tagged words out of 200 total words. Calculate accuracy %.
 8. Name two approaches to POS tagging (rule-based vs. stochastic) and state one difference.
 9. What tag would "quickly" most likely receive in Penn Treebank tagset?
 10. Why does context matter more than dictionary lookup alone in POS tagging?
-

5. Learning to Classify Text

5.1 Concept in Simple Language

Text classification assigns predefined labels/categories to text (e.g., spam/not-spam, positive/negative sentiment) using **supervised machine learning**. The model learns from labeled training data by extracting **features** (word presence, length, POS patterns) and uses algorithms like **Naive Bayes**, **Decision Trees**, or **Maximum Entropy** classifiers.

5.2 Key Definitions

Term	Definition
Feature Extraction	Converting text into a set of measurable attributes (features) for the classifier
Bag of Words (BoW)	Representation of text as an unordered collection of word counts, ignoring grammar/order
Naive Bayes Classifier	Probabilistic classifier based on Bayes' Theorem, assuming feature independence

Term	Definition
Training Set / Test Set	Data used to train the model / data used to evaluate it
Overfitting	Model performs well on training data but poorly on unseen/test data
Precision	Proportion of correctly predicted positive cases among all predicted positives
Recall	Proportion of correctly predicted positive cases among all actual positives
F1-Score	Harmonic mean of precision and recall
Confusion Matrix	Table showing True Positive (TP), True Negative (TN), False Positive (FP), False Negative (FN)

Key Formulas:

```

Bayes' Theorem: P(Label|Features) = [P(Features|Label) × P(Label)] /
P(Features)

Precision = TP / (TP + FP)
Recall    = TP / (TP + FN)
F1-Score  = 2 × (Precision × Recall) / (Precision + Recall)
Accuracy  = (TP + TN) / (TP + TN + FP + FN)

```

5.3 Worked Examples

Example 1: Spam classification feature: `contains_word('free') = True` → higher probability of "spam" label under Naive Bayes.

Example 2: Given: TP=40, FP=10, FN=5, TN=45.

- Precision = 40/(40+10) = 0.80
- Recall = 40/(40+5) = 0.89
- F1 = 2×(0.80×0.89)/(0.80+0.89) ≈ 0.84

Example 3: Gender classification from names: feature = last letter of name (e.g., names ending in 'a' more often female) — a simple but effective feature used in NLTK's classic example.

Example 4: Bag-of-Words for "I love NLP, I love Python" → {I:2, love:2, NLP:1, Python:1} — order is ignored, only counts matter.

Example 5: Overfitting example: A classifier that memorizes training reviews performs 99% on training data but only 60% on new/unseen reviews — indicates poor generalization.

5.4 Practice Questions

1. Naive Bayes classifier is based on: (a) Decision boundaries (b) Bayes' Theorem with independence assumption (c) Gradient descent (d) Clustering
 2. If TP=30, FP=5, FN=10, Recall equals: (a) 0.75 (b) 0.86 (c) 0.60 (d) 0.90
 3. F1-score is the _____ mean of precision and recall.
 4. What does "Bag of Words" ignore that could be linguistically important?
 5. Define overfitting in the context of text classification.
 6. In a spam/ham classifier, a False Positive means: (a) spam correctly identified (b) ham wrongly marked as spam (c) spam missed (d) ham correctly identified
 7. Compute Accuracy given TP=50, TN=40, FP=5, FN=5.
 8. Name two commonly used feature types in text classification.
 9. Why is a separate test set needed apart from the training set?
 10. Which probabilistic theorem underlies the Naive Bayes classifier? State it.
-

6. Deep Learning for NLP

6.1 Concept in Simple Language

Traditional ML needs hand-crafted features; **Deep Learning** automatically learns features/representations from raw text using neural networks. Words are converted into dense numeric vectors (**embeddings**) that capture meaning, and sequential models (**RNN**, **LSTM**) or attention-based models (**Transformers**) process word order and context to perform tasks like translation, sentiment analysis, and text generation.

6.2 Key Definitions

Term	Definition
Word Embedding	A dense vector representation of a word capturing semantic meaning (e.g., Word2Vec, GloVe)
Word2Vec	Neural model learning embeddings via CBOW (predict word from context) or Skip-gram (predict context from word)
RNN (Recurrent Neural Network)	Neural network with loops, processes sequences by maintaining a hidden state
Vanishing Gradient Problem	Gradients shrink during backpropagation through long sequences, hindering learning of long-term dependencies (common issue in vanilla RNNs)
LSTM (Long Short-Term Memory)	RNN variant with gates (input, forget, output) that solves vanishing gradient problem, retains long-term context

Term	Definition
Attention Mechanism	Allows a model to focus on relevant parts of input sequence when producing each output
Transformer	Architecture based entirely on self-attention (no recurrence), enabling parallel processing — basis of BERT, GPT
Embedding Dimension	The size of the vector used to represent each word (e.g., 100, 300)

Key Formula (Cosine Similarity — used to compare word embeddings):

$$\text{Cosine Similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

Value ranges from -1 (opposite) to 1 (identical direction/meaning).

6.3 Worked Examples

Example 1: Word2Vec captures analogies: $\text{vector('King')} - \text{vector('Man')} + \text{vector('Woman')} \approx \text{vector('Queen')}$ — demonstrating embeddings capture semantic relationships.

Example 2: Cosine similarity: If $\text{vector}(\text{cat})$ and $\text{vector}(\text{dog})$ point in similar directions (both "animals"), cosine similarity ≈ 0.8 (high); $\text{vector}(\text{cat})$ and $\text{vector}(\text{car}) \rightarrow$ cosine similarity ≈ 0.1 (low, unrelated).

Example 3: LSTM in sentiment analysis: for the sentence "The movie was not good at all", an LSTM retains the word "not" in memory across the sequence to correctly flip sentiment to negative, unlike a simple bag-of-words model.

Example 4: Attention mechanism in translation: translating "The cat sat on the mat" to French — the model "attends" strongly to "cat" when generating "chat", regardless of position in sentence.

Example 5: Transformer-based model (like BERT) reads a sentence bidirectionally (both left-to-right and right-to-left context) to understand a word's meaning, e.g., disambiguating "bank" (river vs. financial) using full sentence context.

6.4 Practice Questions

- Word2Vec has two main architectures: (a) LSTM & GRU (b) CBOW & Skip-gram (c) Encoder & Decoder (d) RNN & CNN
- Which problem does LSTM primarily solve compared to vanilla RNN? (a) Overfitting (b) Vanishing gradient (c) Underfitting (d) Tokenization errors
- Cosine similarity value of 1 indicates: (a) opposite vectors (b) orthogonal/unrelated vectors (c) identical direction (similar meaning) (d) zero vector

4. What is the key architectural innovation in Transformers that removes the need for recurrence?
 5. Define "word embedding" and name one popular embedding technique.
 6. In LSTM, name the three types of gates and their general purpose.
 7. Why do word embeddings outperform one-hot encoding for representing words?
 8. Which famous deep learning model (based on Transformers) is bidirectional and widely used for NLP tasks?
 9. Give one real-world NLP application of attention mechanisms.
 10. State one advantage and one disadvantage of deep learning approaches compared to traditional ML for NLP.
-

7. Extracting Information from Text

7.1 Concept in Simple Language

Information Extraction (IE) is the task of automatically pulling structured information (names, places, relationships) from unstructured text. Key sub-tasks include **Named Entity Recognition (NER)** — identifying names of people, organizations, locations — and **Relation Extraction** — identifying how entities are related.

7.2 Key Definitions

Term	Definition
Named Entity Recognition (NER)	Identifying and classifying named entities in text (PERSON, ORG, GPE, DATE, etc.)
Chunking	Grouping words into meaningful phrases (e.g., Noun Phrase chunks) without full parsing
IOB Tagging	Labeling scheme for chunks: I=Inside, O=Outside, B=Beginning of a chunk
Relation Extraction	Identifying semantic relationships between recognized entities (e.g., "works for", "located in")
Chinking	Removing a sequence of tokens from an existing chunk
Named Entity Types (common)	PERSON, ORGANIZATION, GPE (Geo-Political Entity), DATE, TIME, MONEY

Chunk Grammar Example (Regex-style rule):

NP: {<DT>?<JJ>*<NN>}

(Optional determiner + zero or more adjectives + a noun = Noun Phrase)

7.3 Worked Examples

Example 1: NER on "Ratan Tata founded companies in Mumbai in 1991." → Ratan
Tata=PERSON, Mumbai=GPE, 1991=DATE

Example 2: Chunking "the big red ball" using rule NP: {<DT>?<JJ>*<NN>} → whole phrase forms one NP chunk: [the/DT big/JJ red/JJ ball/NN].

Example 3: IOB tagging for "The big ball rolled": The/B-NP big/I-NP ball/I-NP rolled/O — B marks the start of the NP chunk, I continues it, O is outside any chunk.

Example 4: Relation extraction: From "Ravi works for Infosys", the extracted relation triple is (Ravi, works_for, Infosys).

Example 5: Chinking example: Given chunk [the/DT quick/JJ brown/JJ fox/NN jumps/VBZ], applying a chunk rule to remove VBZ gives chunk [the/DT quick/JJ brown/JJ fox/NN] with jumps/VBZ outside.

7.4 Practice Questions

1. NER stands for: (a) Named Entity Retrieval (b) Named Entity Recognition (c) Noun Entity Rule (d) None
2. In IOB tagging, "B" denotes: (a) Beginning of chunk (b) Break (c) Boundary word (d) Base form
3. GPE tag in NER typically refers to: (a) Person names (b) Geo-Political Entities (c) Dates (d) Monetary values
4. What is the difference between chunking and full parsing?
5. Define "chinking" with a simple example.
6. Write a simple chunk grammar rule to capture a Noun Phrase (Determiner + Adjective* + Noun).
7. Extract entities from: "Google opened an office in Hyderabad in 2005." (identify ORG, GPE, DATE)
8. What is meant by "Relation Extraction" in IE? Give one example triple.
9. Why is IE considered a step beyond simple POS tagging?
10. Name two real-world applications of Named Entity Recognition.

8. Analysing Sentence Structure

8.1 Concept in Simple Language

Sentence structure analysis (**syntactic parsing**) studies how words combine into phrases

and sentences following grammar rules. **Context-Free Grammar (CFG)** defines valid sentence structures using production rules, and **parsers** use these rules to build **parse trees** showing hierarchical sentence structure.

8.2 Key Definitions

Term	Definition
Context-Free Grammar (CFG)	A set of production rules describing how symbols (S, NP, VP) can be replaced/expanded to generate valid sentences
Parse Tree	A tree diagram representing the syntactic structure of a sentence according to a grammar
Constituency	Grouping of words that function as a single unit (phrase) within a sentence
Dependency Grammar	Represents grammatical relations as direct links (dependencies) between words rather than phrase structure
Ambiguity (Structural)	When a sentence has more than one valid parse tree / interpretation
Recursive Descent Parser	Top-down parser that starts from the start symbol (S) and expands rules to match input
Chart Parser	Uses dynamic programming to efficiently parse using well-formed substring tables, avoiding repeated work

Sample CFG:

```
S  → NP VP
NP → Det N | Det N PP
VP → V NP | V NP PP
PP → P NP
Det → 'the' | 'a'
N   → 'dog' | 'park' | 'man'
V   → 'saw' | 'walked'
P   → 'in' | 'with'
```

8.3 Worked Examples

Example 1: Parse "the dog saw the man" using the CFG above:

```
S
├─ NP → Det(the) N(dog)
└─ VP → V(saw) NP(Det(the) N(man))
```

Example 2: Structural ambiguity: "I saw the man with the telescope" has two parse trees — (1) I used a telescope to see the man (PP attaches to VP) (2) The man had a telescope (PP attaches to NP) — a classic PP-attachment ambiguity example.

Example 3: Dependency parsing of "Ravi eats apples": $\boxed{\text{eats} \rightarrow \text{nsubj} \rightarrow \text{Ravi}}$, $\boxed{\text{eats} \rightarrow \text{dobj} \rightarrow \text{apples}}$ — shows direct grammatical relations rather than nested phrases.

Example 4: Generating sentences from CFG: Using the grammar above, we can also generate "a man walked in the park" by choosing different terminal expansions.

Example 5: Recursive descent parsing "the man saw a dog" fails fast if grammar has no rule matching a token sequence, backtracking to try alternative rules — illustrating top-down, rule-driven matching.

8.4 Practice Questions

1. A Context-Free Grammar rule like $\boxed{\text{NP} \rightarrow \text{Det N}}$ means: (a) NP is replaced by nothing (b) NP can be expanded into a Determiner followed by a Noun (c) N always precedes Det (d) None
 2. The sentence "I saw the man with the telescope" is a classic example of: (a) Lexical ambiguity (b) Structural (PP-attachment) ambiguity (c) Morphological ambiguity (d) No ambiguity
 3. Dependency grammar primarily represents: (a) Phrase structure trees (b) Direct grammatical relations between word pairs (c) Word frequencies (d) POS tags only
 4. Define "parse tree" in your own words.
 5. Write a CFG rule for VP that allows a verb followed by a noun phrase.
 6. What distinguishes a Recursive Descent Parser from a Chart Parser?
 7. Give an example of a sentence with lexical ambiguity (word with multiple meanings) versus structural ambiguity.
 8. In the CFG given in this section, derive the parse for "a man saw a dog" (write $S \rightarrow \text{NP VP}$ expansion steps).
 9. Why do chart parsers avoid the inefficiency of recursive descent parsers on ambiguous grammars?
 10. State one real-world NLP application that relies heavily on syntactic parsing (e.g., grammar checking, machine translation).
-

9. QUICK REVISION FORMULA SHEET

Concept	Formula/Rule
Lexical Diversity	Types / Tokens
Tagging Accuracy	$(\text{Correct tags} / \text{Total words}) \times 100$
Precision	$TP / (TP + FP)$
Recall	$TP / (TP + FN)$
F1-Score	$2 \times (P \times R) / (P + R)$
Accuracy	$(TP+TN) / (TP+TN+FP+FN)$
Bayes' Theorem	$P(L F) = P(F L) \cdot P(L) / P(F)$
Cosine Similarity	$(A \cdot B) / (A B)$
Stemming vs Lemmatization	Stemming = rule-based, crude; Lemmatization = dictionary-based, accurate
CFG Rule Format	LHS $\rightarrow$ RHS (e.g., $S \rightarrow NP VP$)
IOB Tags	B = Begin chunk, I = Inside chunk, O = Outside

10. MIXED MOCK TEST (10 Questions Across All Topics)

1. If a text has 300 tokens and 75 types, its lexical diversity is _____.
2. Which lexical resource organizes English words into synsets connected by hypernym/hyponym relations?
3. Convert to lemma: "went" (verb, past tense of "go") using WordNet lemmatizer with correct POS.
4. Which Penn Treebank tag is assigned to a plural noun?
5. A classifier gives TP=60, FP=15, FN=5. Find Precision and Recall.
6. What is the main advantage of LSTM over vanilla RNN in NLP tasks?
7. Identify the named entities in: "Narayana Murthy co-founded Infosys in Pune in 1981."
8. Write a simple CFG rule to generate a prepositional phrase (PP $\rightarrow$ P NP).
9. Explain, with an example, one case of PP-attachment ambiguity.
10. State the two Word2Vec training architectures and briefly describe the difference between them.

Revision Tips

- Revise the **definitions table** in each section a day before the exam — most MCQs test definitions and small distinctions (stemming vs lemmatization, precision vs recall, hypernym vs hyponym).
- Practice **manual calculation** of lexical diversity, precision/recall/F1, and cosine similarity — numeric questions are common and scoring.
- Draw at least one **parse tree** and one **CFG derivation** by hand until comfortable — visualization helps retention.
- Make your own flash cards for **Penn Treebank tags** and **IOB tagging** — these are frequently tested via short factual questions.
- For Deep Learning for NLP, focus conceptually on **why** each model (RNN → LSTM → Transformer) was invented to solve the previous model's limitation — exams often test this progression/reasoning.

All the best for TG CPGET 2026!